

PYTHON

COMPLETE NOTES

Easy Language • Simple Examples • Important Topics Highlighted

★ Red star = Important Topic (read carefully, exam favorite)

MMOC TEACH
MADE BY PRAVENDRA

INDEX

★ 1. Introduction to Python

★ 2. Variables and Data Types

- 3. Type Casting (Changing Data Type)

★ 4. Operators

- 5. Taking Input from User

★ 6. Conditional Statements (if, elif, else)

★ 7. Loops - for and while

- 8. Break, Continue and Pass

★ 9. Strings

★ 10. Lists

- 11. Tuples
- 12. Sets

★ 13. Dictionaries

★ 14. Functions

- 15. *args and **kwargs
- 16. Lambda Functions
- 17. List and Dictionary Comprehension
- 18. Modules and Packages

★ 19. File Handling

★ 20. Exception Handling (try, except)

★ 21. Object-Oriented Programming (OOP) - Classes and Objects

★ 22. Inheritance

- 23. Polymorphism
- 24. Encapsulation and Abstraction
- 25. Iterators and Generators
- 26. Decorators
- 27. Regular Expressions (Basics)

★ 1. Introduction to Python [IMPORTANT TOPIC]

Explanation:

Python is a simple, easy-to-read programming language. It is used to make websites, apps, games, and to work with data and Artificial Intelligence.

Python code looks almost like normal English, so it is a very good language for beginners.

Python is called an 'interpreted' language. This means the computer reads and runs the code line by line, instead of converting the whole program first.

Key Points:

- Easy to learn and read.
- Free and open source.
- Works on Windows, Mac, and Linux.
- Has a huge collection of ready-made libraries (tools).

Example Code:

```
# This is your first Python program  
print("Hello, World!")
```

Output:

```
Hello, World!
```

💡 *Note: The print() function is used to show text on the screen. You will use it very often.*

★ 2. Variables and Data Types [IMPORTANT TOPIC]

Explanation:

A variable is like a box in which we store a value, so we can use it later. In Python, we do not need to tell the computer what type of value it is — Python understands it automatically.

Every value in Python has a 'data type', which tells what kind of value it is: a number, text, true/false, etc.

Key Points:

- int -> whole numbers, example: 10, -5
- float -> decimal numbers, example: 3.14
- str -> text (string), example: "hello"
- bool -> True or False

Example Code:

```
name = "Pravendra"    # str
age = 21               # int
height = 5.9           # float
is_student = True      # bool

print(name, age, height, is_student)
print(type(age))
```

Output:

```
Pravendra 21 5.9 True
<class 'int'>
```

 *Note: You can check the data type of any variable using the type() function.*

3. Type Casting (Changing Data Type)

Explanation:

Sometimes we need to change one data type into another. This is called type casting. For example, changing a number written as text into a real number.

Example Code:

```
a = "25"
b = int(a)      # string -> int
print(b + 5)

c = 9.8
print(int(c))   # float -> int (removes decimal part)
```

Output:

```
30
9
```

 *Note: int(), float(), str() are the most common casting functions.*

★ 4. Operators [IMPORTANT TOPIC]

Explanation:

Operators are special symbols that perform an action on values. For example, + adds two numbers, and > checks which number is bigger.

Key Points:

- Arithmetic: + - * / // % ** (add, subtract, multiply, divide, floor divide, remainder, power)
- Comparison: == != > < >= <=
- Logical: and, or, not
- Assignment: = += -= *= /=

Example Code:

```
a = 10
b = 3
print(a + b)    # 13
print(a % b)    # 1 (remainder)
print(a > b and b > 0)    # True
```

Output:

```
13
1
True
```

5. Taking Input from User

Explanation:

The `input()` function is used to take information from the user while the program is running. Whatever the user types is always received as a string (text).

Example Code:

```
name = input("Enter your name: ")
age = int(input("Enter your age: "))
print("Hello", name, "- Next year you will be", age + 1)
```

Output:

```
Enter your name: Pravendra
Enter your age: 21
Hello Pravendra - Next year you will be 22
```

 *Note: Always convert `input()` to `int()` or `float()` if you plan to do maths with it.*

★ 6. Conditional Statements (if, elif, else) [IMPORTANT TOPIC]

Explanation:

Conditional statements help the program make decisions. The code inside 'if' runs only when the condition is True. 'elif' means 'else if' — it checks another condition. 'else' runs when none of the above conditions are true.

Example Code:

```
marks = 75

if marks >= 90:
    print("Grade A")
elif marks >= 60:
    print("Grade B")
else:
    print("Grade C")
```

Output:
Grade B

 *Note: Indentation (spaces at the start of a line) is very important in Python. It shows which code belongs to the if/else block.*

★ 7. Loops - for and while [IMPORTANT TOPIC]

Explanation:

A loop is used to repeat a block of code many times, instead of writing it again and again.

A 'for' loop is used when we know how many times to repeat, or when we go through items one by one (like a list).

A 'while' loop repeats as long as a condition stays True.

Example Code:

```
# for loop
for i in range(1, 4):
    print("For loop count:", i)

# while loop
n = 1
while n <= 3:
    print("While loop count:", n)
    n = n + 1
```

Output:

```
For loop count: 1
For loop count: 2
For loop count: 3
While loop count: 1
While loop count: 2
While loop count: 3
```

 Note: `range(1, 4)` gives numbers 1, 2, 3 (the last number 4 is not included).

8. Break, Continue and Pass

Explanation:

'break' stops the loop completely. 'continue' skips the current step and moves to the next one. 'pass' does nothing — it is used as a placeholder.

Example Code:

```
for i in range(1, 6):  
    if i == 3:  
        continue    # skip 3  
    if i == 5:  
        break        # stop at 5  
    print(i)
```

Output:

```
1  
2  
4
```

★ 9. Strings [IMPORTANT TOPIC]

Explanation:

A string is text written inside quotes (" " or ' '). Strings are one of the most used data types because almost every program deals with text.

Strings in Python have many built-in functions (called methods) that help us change or check the text easily.

Key Points:

- `len(s)` -> length of the string
- `s.upper()` / `s.lower()` -> change case
- `s.strip()` -> remove extra spaces
- `s.replace('a','b')` -> replace text
- `s.split(',')` -> break string into a list
- Slicing: `s[0:3]` -> get part of the string

Example Code:

```
text = " Hello Python "  
print(text.strip())  
print(text.upper())  
print(len(text))  
print(text.strip()[0:5])
```

Output:

```
Hello Python  
HELLO PYTHON  
16  
Hello
```

★ 10. Lists [IMPORTANT TOPIC]

Explanation:

A list is used to store many values together in one variable, in a fixed order. Lists are written using square brackets []. Lists can be changed after creation (we can add, remove or update items) — this is called 'mutable'.

Key Points:

- `append(x)` -> add item at the end
- `remove(x)` -> remove a specific item
- `pop()` -> remove last item
- `sort()` -> arrange items in order
- `len(list)` -> number of items

Example Code:

```
fruits = ["apple", "banana", "mango"]
fruits.append("orange")
fruits.remove("banana")
print(fruits)
print(fruits[0])      # first item
print(len(fruits))
```

Output:

```
['apple', 'mango', 'orange']
apple
3
```

💡 Note: List index always starts from 0, not 1.

11. Tuples

Explanation:

A tuple is similar to a list, but once created, it cannot be changed. This is called 'immutable'. Tuples are written using round brackets (). We use tuples when we want the data to stay fixed and safe from accidental changes.

Example Code:

```
point = (10, 20)
print(point[0])
print(point[1])
# point[0] = 15    # This will give an ERROR because tuples cannot be changed
```

Output:

```
10
20
```

12. Sets

Explanation:

A set stores multiple values like a list, but it never allows duplicate (repeated) values, and the order of items is not fixed. Sets are written using curly brackets { }.

Example Code:

```
numbers = {1, 2, 2, 3, 3, 4}
print(numbers)      # duplicates removed automatically
numbers.add(5)
print(numbers)
```

Output:

```
{1, 2, 3, 4}
{1, 2, 3, 4, 5}
```

★ 13. Dictionaries [IMPORTANT TOPIC]

Explanation:

A dictionary stores data as 'key : value' pairs, similar to a real dictionary where a word (key) has a meaning (value). Dictionaries are written using curly brackets { }. They are very useful for storing related information, like a student's details.

Example Code:

```
student = {  
    "name": "Pravendra",  
    "age": 21,  
    "course": "Python"  
}  
  
print(student["name"])  
student["age"] = 22      # update value  
student["city"] = "Delhi" # add new key  
print(student)
```

Output:

```
Pravendra  
{'name': 'Pravendra', 'age': 22, 'course': 'Python', 'city': 'Delhi'}
```

💡 *Note: Use .get('key') to safely read a value without causing an error if the key does not exist.*

★ 14. Functions [IMPORTANT TOPIC]

Explanation:

A function is a block of reusable code that performs one specific task. Instead of writing the same code again and again, we write it once inside a function and 'call' it whenever needed.

A function is created using the 'def' keyword. It can accept inputs (called parameters) and can send back a result using 'return'.

Example Code:

```
def add_numbers(a, b):  
    result = a + b  
    return result  
  
total = add_numbers(5, 7)  
print("Total is:", total)
```

Output:

Total is: 12

 *Note: A function only runs when it is called. Just defining it does nothing by itself.*

15. *args and **kwargs

Explanation:

Sometimes we don't know in advance how many values will be passed to a function. *args lets a function accept any number of normal values (as a tuple). **kwargs lets a function accept any number of named values (as a dictionary).

Example Code:

```
def show_all(*args, **kwargs):  
    print("Args:", args)  
    print("Kwargs:", kwargs)  
  
show_all(1, 2, 3, name="Pravendra", age=21)
```

Output:

```
Args: (1, 2, 3)  
Kwargs: {'name': 'Pravendra', 'age': 21}
```

16. Lambda Functions

Explanation:

A lambda function is a short, one-line function without a name (also called an anonymous function). It is used when we need a simple function for a short time.

Example Code:

```
square = lambda x: x * x
print(square(5))

add = lambda a, b: a + b
print(add(4, 6))
```

Output:

```
25
10
```

17. List and Dictionary Comprehension

Explanation:

Comprehension is a short and fast way to create a new list or dictionary in just one line, instead of writing a full loop with multiple lines.

Example Code:

```
# normal way
squares = []
for i in range(1, 6):
    squares.append(i * i)

# comprehension way (does the same thing in 1 line)
squares2 = [i * i for i in range(1, 6)]
print(squares2)
```

Output:

```
[1, 4, 9, 16, 25]
```

18. Modules and Packages

Explanation:

A module is simply a Python file (.py) containing code that we can reuse in another file. A package is a folder containing many modules. Python also comes with many built-in modules that we can import and use directly.

Example Code:

```
import math
print(math.sqrt(16))
print(math.pi)

import random
print(random.randint(1, 10))
```

Output:

```
4.0
3.141592653589793
(a random number between 1 and 10)
```

★ 19. File Handling [IMPORTANT TOPIC]

Explanation:

File handling means reading data from a file or writing data into a file using Python. This is useful for saving information permanently, even after the program stops running.

Key Points:

- 'w' -> write (creates new / overwrites file)
- 'r' -> read the file
- 'a' -> append (add data at the end)
- Always close the file after use, or use 'with' to do it automatically.

Example Code:

```
# Writing to a file
with open("notes.txt", "w") as f:
    f.write("Hello, this is my file.")

# Reading from a file
with open("notes.txt", "r") as f:
    content = f.read()
    print(content)
```

Output:

```
Hello, this is my file.
```

 *Note: Using 'with open(...) as f:' is the safest way — it closes the file automatically, even if an error occurs.*

★ 20. Exception Handling (try, except) [IMPORTANT TOPIC]

Explanation:

Sometimes a program crashes because of an unexpected error (like dividing by zero). Exception handling lets us 'catch' this error and handle it safely, instead of the whole program stopping suddenly.

Example Code:

```
try:
    num = int(input("Enter a number: "))
    result = 10 / num
    print(result)
except ZeroDivisionError:
    print("Error: You cannot divide by zero!")
except ValueError:
    print("Error: Please enter a valid number!")
finally:
    print("Program finished running.")
```

Output:

```
Enter a number: 0
Error: You cannot divide by zero!
Program finished running.
```

💡 *Note: 'finally' block always runs, whether there is an error or not.*

★ 21. Object-Oriented Programming (OOP) - Classes and Objects

[IMPORTANT TOPIC]

Explanation:

OOP is a way of writing programs using 'objects', which represent real-world things (like a Car, a Student, a Bank Account).

A class is a blueprint/design for creating objects. An object is a real item created from that class. Every class usually has a special method called `__init__`, which runs automatically when a new object is created.

Example Code:

```
class Student:
    def __init__(self, name, marks):
        self.name = name
        self.marks = marks

    def show_details(self):
        print(self.name, "scored", self.marks, "marks")

s1 = Student("Pravendra", 90)
s1.show_details()
```

Output:

Pravendra scored 90 marks

 *Note: 'self' refers to the current object itself. It must be the first parameter in every method inside a class.*

★ 22. Inheritance [IMPORTANT TOPIC]

Explanation:

Inheritance allows one class (child class) to use the properties and methods of another class (parent class). This helps us reuse code instead of writing it again.

Example Code:

```
class Animal:
    def speak(self):
        print("This animal makes a sound")

class Dog(Animal):          # Dog inherits from Animal
    def bark(self):
        print("The dog barks: Woof!")

d = Dog()
d.speak()    # inherited from Animal
d.bark()     # defined in Dog
```

Output:

```
This animal makes a sound
The dog barks: Woof!
```

23. Polymorphism

Explanation:

Polymorphism means 'many forms'. It allows the same method name to behave differently depending on which object is using it.

Example Code:

```
class Cat:
    def sound(self):
        print("Cat says Meow")

class Cow:
    def sound(self):
        print("Cow says Moo")

for animal in [Cat(), Cow()]:
    animal.sound()
```

Output:

```
Cat says Meow
Cow says Moo
```

24. Encapsulation and Abstraction

Explanation:

Encapsulation means hiding the internal details of a class and only allowing controlled access to the data (usually using an underscore _ or double underscore __ before the variable name).

Abstraction means showing only the necessary/important details to the user, and hiding the complex working behind the scenes.

Example Code:

```
class BankAccount:
    def __init__(self, balance):
        self.__balance = balance    # private variable

    def show_balance(self):
        print("Your balance is:", self.__balance)

acc = BankAccount(5000)
acc.show_balance()
# print(acc.__balance)    # This will give an ERROR - it is private
```

Output:

```
Your balance is: 5000
```

25. Iterators and Generators

Explanation:

An iterator is an object that lets us go through a collection of items one by one. A generator is a simple way to create an iterator using a function with the 'yield' keyword instead of 'return'. Generators are memory-efficient because they produce one value at a time, instead of storing everything at once.

Example Code:

```
def count_up_to(n):  
    num = 1  
    while num <= n:  
        yield num  
        num += 1  
  
for value in count_up_to(4):  
    print(value)
```

Output:

```
1  
2  
3  
4
```

26. Decorators

Explanation:

A decorator is a special function that adds extra features to another function, without changing its original code. Decorators are written using the @ symbol above a function.

Example Code:

```
def my_decorator(func):  
    def wrapper():  
        print("Before the function runs")  
        func()  
        print("After the function runs")  
    return wrapper  
  
@my_decorator  
def greet():  
    print("Hello!")  
  
greet()
```

Output:

```
Before the function runs  
Hello!  
After the function runs
```

27. Regular Expressions (Basics)

Explanation:

Regular Expressions (regex) are patterns used to search, match, or check text — for example, checking if an email address is written correctly. Python has a built-in module called 're' for this.

Example Code:

```
import re

text = "My email is test@example.com"
pattern = r"[\w.]+@[ \w. ]+"

match = re.search(pattern, text)
print(match.group())
```

Output:

```
test@example.com
```